CRIMS
City Resource & Infrastructure Management

Sprint 2 & Sprint 3 — Story Plan

Sprint 2 → Review 3 | Sprint 3 → Review 4

Project CRIMS / InfraLynx
Institution Sri City — FFSD Course, S2026
JIRA Project SCRUM (Infra-Lynx) · infralynx-team.atlassian.net
Prepared by Yshasvee Patel (Team Lead)
Team Yshasvee Patel · Raveendra · Pankaj · Raman · Karthik
Version v1.0 — March 2026

Sprint 1 Status — Review 2 Deliverable

Sprint-1 (Mar 02 – Mar 09, 2026) · Active Sprint

Goal: establish a complete design foundation for CRIMS — Figma screens for all 3 actor modules, relational DB schema, ER diagram, and GitHub repository setup. All major stories are Done. SCRUM-87 (ER diagram, Raveendra) remains In Progress and must be closed before Sprint 2 work begins.

SCRUM ID	Summary	Assignee	Status	SP
SCRUM-85	Login page with role selection and RBAC enforcement	Pankaj	Done	5
SCRUM-86	Design and document the relational database schema for CRIMS	Karthik	Done	13
SCRUM-87	Create ER diagram for the CRIMS database	Raveendra	In Progress	15
SCRUM-88	Set up GitHub repository structure and push all project artifacts	Karthik	Done	8
SCRUM-92	Design Dept Officer Dashboard and Task Management screens (Figma)	Yshasvee	Done	8
SCRUM-93	Design Dept Officer Maintenance, Field Teams and Budget Verification screens	Yshasvee	Done	8
SCRUM-94	Work order review and approval workflow for City Administrator	Pankaj	Done	8
SCRUM-95	Task assignment module for Department Officer to delegate to Field Engineers	Yshasvee	Done	8
SCRUM-98	Vendor quotation comparison and GST compliance verification module	Yshasvee	Done	13
SCRUM-100	City-wide resource analytics, department ranking and report generation	Pankaj	Done	8
SCRUM-101	Completed work validation and QC certification workflow	Pankaj	Done	13
SCRUM-102	Field Engineer home dashboard — task summary, emergency alert, notifications	Raman	Done	8
SCRUM-103	Field Engineer Execute Work Orders screen with task cards and progress tracking	Raman	Done	8
SCRUM-104	Field Engineer Prepare Reports form and Profile & Settings screen	Raman	Done	5

Sprint 2 — Review 3 Deliverable

Sprint Goal	JS-enabled frontend with client-side validation + Express MVC backend with in-memory mock data + frontend pages wired to backend routes via fetch().
Sprint Dates	Mar 10, 2026 → Mar 23, 2026 (2 weeks)
Review Target	Review 3 — JS frontend · client-side validation · Express MVC · routes with mock data
Total Stories	17 stories
Story Points	65 SP
JIRA Sprint Name	Sprint-2 Review 3 Deliverable

Review 3 Deliverables (from syllabus Week 10)

1) JS-enabled frontend 2) Client-side validation 3) Express project structure 4) MVC folders 5) Routes with mock data

A. JavaScript & Client-Side Validation (7 stories · 24 SP)

Adds interactivity and form validation to the static HTML pages from Sprint 1. No server calls at this stage — pure JavaScript and DOM manipulation.

#	Story ID	Story Title & Description	Type	Assignee	Priority	SP
1	S2-01	**JS — Login page role selection and form validation** Role card toggle (one at a time, border highlights green on selection), email regex validation, password min 8 chars, inline error messages per field. Submit button disabled until all fields valid. Simulated role-based redirect on success.	Frontend	Pankaj	High	3
2	S2-02	**JS — Work Order approve/reject modal with DOM update (City Admin)** Approve modal requires justification textarea (min 20 chars). Reject modal requires a reason. On confirm, table row status badge updates via DOM — no page reload. Modal closes on cancel or outside click.	Frontend	Raveendra	High	4
3	S2-03	**JS — Budget/Vendor form validation with GST regex (Dept Officer)** GSTIN validated against Indian 15-char alphanumeric regex (e.g. 37AAAAA0000A1Z5). Unit cost must be positive. Minimum 3 vendor entries required before submission. Red danger indicator on rows with missing or invalid GSTIN.	Frontend	Pankaj	High	4

#	Story ID	Story Title & Description	Type	Assignee	Priority	SP
4	S2-04	**JS — Inspection/Report Issue form validation (Field Engineer)** Asset ID required, GPS field required, condition rating 1-5 must be selected, description min 30 chars, at least one photo attached. Inline error message per field. Submit button disabled until all validations pass.	Frontend	Raman	High	4
5	S2-05	**JS — Client-side table filter and search on Work Order / Task tables** Search bar filters rows by WO ID or asset name (case-insensitive). Status dropdown filters by PENDING / APPROVED / IN_PROGRESS / COMPLETED. Priority filter for HIGH / MEDIUM / LOW. Pure DOM filtering — no page reload.	Frontend	Raman	Medium	3
6	S2-06	**JS — Dynamic dashboard stat cards from mock data (City Admin)** Populate KPI stat cards via a JS mock data object (38 Active Work Orders, Rs 18,40,000 utilised, 14 Field Engineers, 5 Pending Quotations). Add CSS transition animation on page load for the budget utilisation bar.	Frontend	Karthik	Medium	3
7	S2-07	**JS — Task assignment modal with workload warning (Dept Officer)** Assign-task modal lists field engineers with current active task count. Engineers with more than 3 active tasks trigger a yellow warning banner. On confirm, table row Assignee column updates via DOM without page reload.	Frontend	Yshasvee	Medium	3

B. Express.js Backend, MVC Setup & Frontend Integration (10 stories · 41 SP)

Establishes the Node.js + Express backend, creates the in-memory mock data store with Sri City sample data, builds all role-based API routes, and wires frontend pages to the backend via fetch() calls.

#	Story ID	Story Title & Description	Type	Assignee	Priority	SP
1	S2-08	**Express — Project initialisation and MVC folder structure** Initialise Node.js + Express: package.json, express/nodemon/dotenv. Create MVC folder tree (/models, /views, /controllers, /routes, /public, /middleware). Add .gitignore, app.js entry point. Update README with local dev setup instructions.	Backend	Yshasvee	High	3
2	S2-09	**Express — In-memory mock data store with Sri City sample data** Create data/mockStore.js: 5 users across 3 roles, 8 work orders (WO-SC-2026-001 to 008) with	Backend	Raman	High	3

#	Story ID	Story Title & Description	Type	Assignee	Priority	SP
		mixed statuses, 3 departments (Roads, Water, Electrical), 6 tasks. Drives all mock route handlers throughout Sprint 2.				
3	S2-10	**Express — Auth route with role-based mock login** POST /api/auth/login validates credentials against mock store and returns user object with role. Role-checking middleware on protected routes. No JWT yet. Test all 3 role logins via Postman. Document sample requests in README.	Backend	Yshasvee	High	4
4	S2-11	**Express — Work Order routes (GET all, GET by ID, POST approve/reject)** GET /api/workorders (all, supports ?status= filter), GET /api/workorders/:id (with task list), POST /api/workorders/:id/approve and /reject. Controller in workOrderController.js, routes in workOrderRoutes.js.	Backend	Raveendra	High	4
5	S2-12	**Express — Task and Department routes** GET /api/tasks (supports ?assignedTo= and ?status= filters), GET /api/tasks/:id, GET /api/departments with budget summary. MVC pattern throughout. Sample INR amounts in budget response (e.g. Rs 45,00,000 allocated).	Backend	Pankaj	High	3
6	S2-13	**Express — Budget and User routes with role-based access control** GET /api/budget/:departmentId (allocation and utilisation), GET /api/users (Admin only — middleware enforced), GET /api/users/:id/tasks (Engineer task list). Verify /api/users returns 403 for non-admin roles.	Backend	Karthik	Medium	3
7	S2-14	**Frontend-Backend — Fetch and render Work Orders dynamically (City Admin)** Replace hardcoded WO table with fetch() to GET /api/workorders. Render rows from JSON (WO ID, asset name, priority badge, status badge). Handle loading spinner and empty state. Wire Approve/Reject to POST routes.	Frontend	Raveendra	High	4
8	S2-15	**Frontend-Backend — Fetch and render Tasks for Officer and Engineer views** Wire Dept Officer task list and Field Engineer task queue to GET /api/tasks with role-based query params. Field Engineer view shows only tasks assigned to the simulated logged-in user. Handle empty state.	Frontend	Pankaj	Medium	4
9	S2-16	**Express — Global error handling middleware** Global handler covering 404 (route not found), 400 (validation error), 500 (internal error). All responses return JSON { status, message, code }. Test with invalid route, missing param, and force-throw.	Backend	Karthik	Medium	2

#	Story ID	Story Title & Description	Type	Assignee	Priority	SP
10	S2-17	**Documentation — README update with MVC structure, API endpoints, run guide** Update README: Express setup, MVC folder tree, all API endpoints (method, path, description, sample response with INR data and WO-SC-2026 IDs), mock credentials for all 3 roles, Postman guide. Commit before Review 3.	Task	Yshasvee	Medium	2

Sprint 3 — Review 4 Deliverable

Sprint Goal	Fully functional CRUD application with SQLite DB integration, end-to-end admin and end-user workflows, integrated frontend-backend, error handling, and complete project documentation.
Sprint Dates	Mar 24, 2026 → Apr 06, 2026 (2 weeks)
Review Target	Review 4 — Full CRUD app · integrated FE+BE · admin & end-user workflows · error handling · README + architecture + API docs
Total Stories	15 stories
Story Points	54 SP
JIRA Sprint Name	Sprint-3 Review 4 Deliverable

Review 4 Deliverables (from syllabus Week 12)
1) Fully functional CRUD application 2) Admin and end-user workflows 3) Integrated frontend and backend 4) Basic error handling 5) Documentation: README, Architecture diagrams, API endpoints

A. CRUD APIs & Database Integration (5 stories · 20 SP)

Replace the in-memory mock store with SQLite. Full Create/Read/Update/Delete for Work Orders, Tasks, Budget Allocations, Vendor Quotations, Inspection Reports, and Resource Requests.

#	Story ID	Story Title & Description	Type	Assignee	Priority	SP
1	S3-01	**DB integration — Replace mock store with SQLite (better-sqlite3)** Install better-sqlite3. Create db.js to initialise DB and run DBSchema.sql on startup. Migrate 8 mock work orders (WO-SC-2026-001 to 008), 5 users, 6 tasks, and 3 departments into the DB. Update all controllers to query DB instead of mockStore.js.	Backend	Karthik	High	5
2	S3-02	**CRUD — Work Order create, update, and delete endpoints** POST /api/workorders (create), PUT /api/workorders/:id (update status/priority/department), DELETE /api/workorders/:id (admin only, soft delete with deleted_at timestamp). Input validation in controller. Test all operations with Postman.	Backend	Raveendra	High	4
3	S3-03	**CRUD — Task create, update, and delete endpoints** POST /api/tasks (create task linked to a WorkOrder), PUT /api/tasks/:id (update status,	Backend	Pankaj	High	4

#	Story ID	Story Title & Description	Type	Assignee	Priority	SP
		progress %, assigned engineer), DELETE /api/tasks/:id (officer only). Validate FK reference to WorkOrder. Test with Sri City sample data.				
4	S3-04	**CRUD — Budget allocation and Vendor Quotation endpoints** POST /api/budget/:departmentId/allocate (Admin allocates budget), POST /api/quotations (Officer adds vendor with GSTIN), PUT /api/quotations/:id (update status). GSTIN required. Validate all INR amount fields are positive.	Backend	Yshasvee	High	4
5	S3-05	**CRUD — Inspection Report and Resource Request endpoints** POST /api/inspections (Field Engineer submits report with condition rating, GPS, photo path), POST /api/resource-requests (urgency level required), GET /api/inspections?workOrderId= (list per WO). Validate all required fields.	Backend	Raman	Medium	3

B. End-to-End Workflows & Frontend-Backend Integration (7 stories · 28 SP)

Connect all frontend pages to live backend endpoints. Demonstrate admin and end-user workflows end-to-end with data persisting in SQLite. Dashboard charts render live data via Chart.js.

#	Story ID	Story Title & Description	Type	Assignee	Priority	SP
1	S3-06	**End-to-end — Full Work Order lifecycle (Admin to Officer to Engineer)** Demonstrate the complete workflow: Admin creates and approves WO, Officer assigns it as a Task to Field Engineer, Engineer updates progress to COMPLETED, Admin validates and sends for QC. All status transitions persist in DB and update live on frontend.	Backend	Yshasvee	High	5
2	S3-07	**End-to-end — Budget allocation and staged fund release workflow** Officer submits min 3 vendor quotations, Admin allocates budget, Officer authorises staged fund release per milestone. Fund release button disabled until milestone is verified. Budget utilisation counter updates live on City Admin dashboard.	Backend	Pankaj	High	5
3	S3-08	**Frontend — City Admin dashboard with live API data and Chart.js charts** Replace all hardcoded dashboard values with live API calls to GET /api/dashboard/stats. Render resource usage chart (water/electricity, 7-month	Frontend	Raman	High	4

#	Story ID	Story Title & Description	Type	Assignee	Priority	SP
		trend), department performance bar chart, and WO trend chart using Chart.js.				
4	S3-09	**Frontend — Full CRUD UI for Work Orders (City Admin)** Admin can create a new WO via form (asset, location, priority, description), edit an existing WO, and soft-delete one. Form submits to POST/PUT /api/workorders. Table refreshes dynamically after each operation. Success and error states shown.	Frontend	Raveendra	High	4
5	S3-10	**Frontend — Full CRUD UI for Tasks and Inspection Reports (Officer and Engineer)** Officer can create tasks from approved WOs, update task status, and reassign engineers. Engineer submits inspection reports and resource requests, views their submissions list. All connected to live backend endpoints.	Frontend	Pankaj	High	4
6	S3-11	**Error handling — Frontend error display and API validation feedback** Display user-friendly error messages for all API responses (400, 403, 404, 500). Toast notifications or inline banners per error type. Full error logged to browser console in dev mode. No raw stack traces exposed to users.	Frontend	Karthik	Medium	3
7	S3-12	**Security — Input sanitisation and rate-limit protection on API routes** Add express-validator (or manual checks) on all POST/PUT routes to strip HTML and enforce field lengths. Add express-rate-limit (100 req/15min per IP) on auth and write routes. Log rate-limit hits to server console.	Backend	Karthik	Medium	3

C. Documentation (3 stories · 6 SP)

All documentation deliverables required for Review 4: final README, system architecture diagram, and complete API endpoint reference document.

#	Story ID	Story Title & Description	Type	Assignee	Priority	SP
1	S3-13	**Documentation — Final README with project overview and setup guide** Complete README: InfraLynx overview and problem statement, team roles, local dev setup (Node version, npm install, DB init with DBSchema.sql), demo credentials for all 3 roles, links to Figma, JIRA board, ER diagram, and GitHub repo structure.	Task	Yshasvee	High	2
2	S3-14	**Documentation — System architecture diagram**	Task	Raman	High	2

#	Story ID	Story Title & Description	Type	Assignee	Priority	SP
		Architecture diagram showing three-tier layout: static HTML frontend — Express MVC backend — SQLite DB. Include role-based access flow (Admin/Officer/Engineer), main API route groups, and external tool links (JIRA, GitHub, Figma). Export as PNG and commit to /docs.				
3	S3-15	**Documentation — Complete API endpoint reference** All API endpoints in a structured table: method, path, description, request body, sample JSON response with INR amounts and WO-SC-2026 IDs, required role. Covers auth, workorders, tasks, departments, budget, quotations, inspections, resource-requests.	Task	Raveendra	Medium	2